第12章 查询优化深度详解

一、查询优化概述

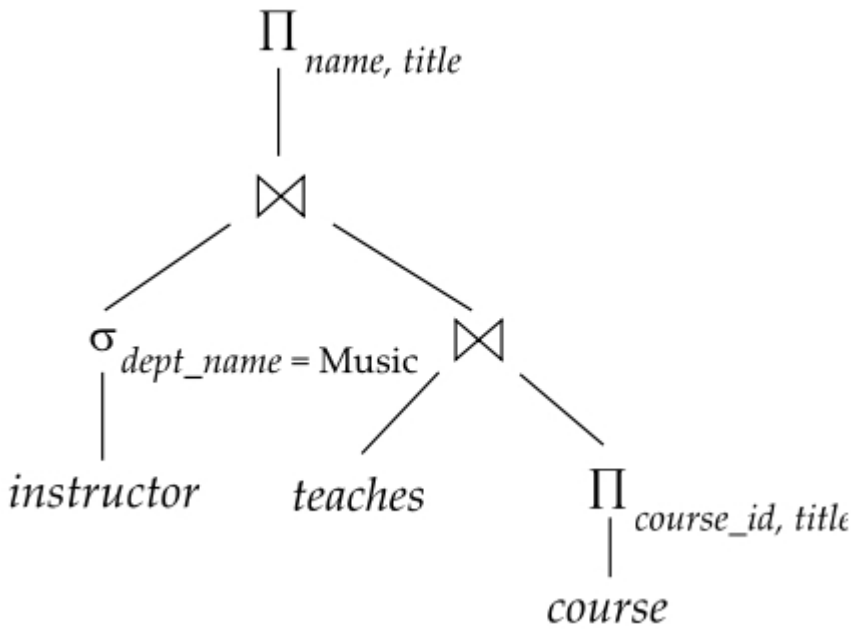
1.1 优化的重要性

查询优化是数据库系统的核心组件，负责将用户查询转换为高效的执行计划。优化质量直接影响查询性能，不同执行计划的代价差异可能达到**数量级**级别（秒级 vs 天级）。

1.2 优化流程

基于成本的查询优化包含三个核心步骤：

1. **生成逻辑等价表达式** ：使用等价规则重写查询
2. **生成替代查询计划** ：为每个表达式标注具体算法
3. **选择最优计划** ：基于成本估计选择代价最小的计划



二、关系代数等价规则深度解析

2.1 等价规则基础概念

两个关系代数表达式等价的条件：在所有合法数据库实例上生成相同的元组集合（顺序无关）。

重要说明 ：SQL中处理的是多重集（multisets），需要考虑重复元组。

2.2 核心等价规则详解

规则1-4：选择和投影的基础规则

```
-- 规则1：合取选择分解
σ_{θ1∧θ2}(E) ≡ σ_{θ1}(σ_{θ2}(E))

-- 规则2：选择交换律
σ_{θ1}(σ_{θ2}(E)) ≡ σ_{θ2}(σ_{θ1}(E))

-- 规则3：投影序列简化
Π_{L1}(Π_{L2}(...(Π_{Ln}(E))...)) ≡ Π_{L1}(E) [当 L1 ⊆ L2 ⊆ ... ⊆ Ln]

-- 规则4：选择与连接结合
σ_θ(E1 × E2) ≡ E1 ⋈_θ E2
```

例题1：规则应用

给定查询：找出工资大于50000的计算机系教师

```
SELECT * FROM instructor
WHERE dept_name = 'Comp. Sci.' AND salary > 50000;
```

应用规则1进行重写：

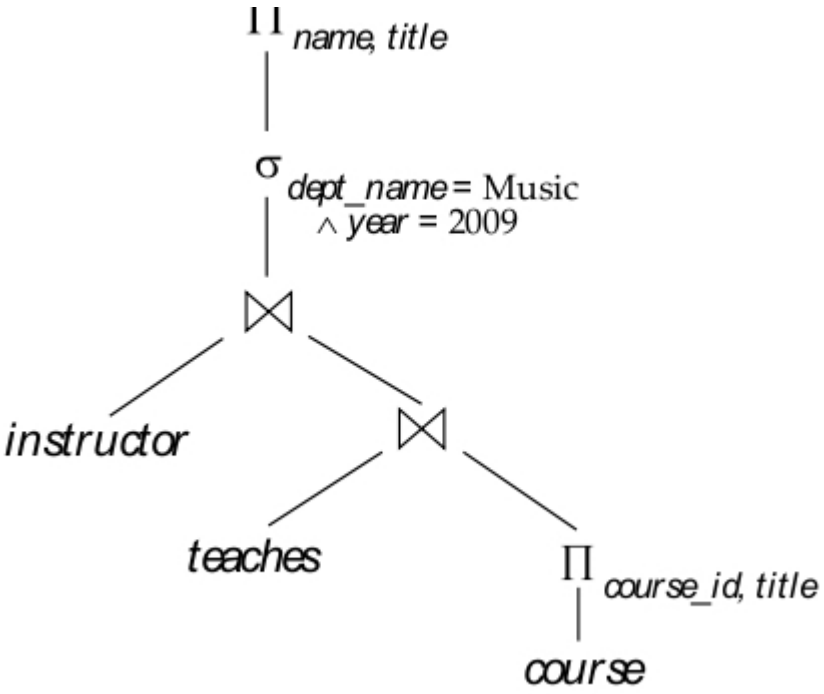
```
σ_{dept_name='Comp. Sci.' ∧ salary>50000}(instructor)
≡ σ_{dept_name='Comp. Sci.'}(σ_{salary>50000}(instructor))
```

规则5-6：连接操作的交换律和结合律

```
-- 规则5：连接交换律
E1 ⋈ E2 ≡ E2 ⋈ E1

-- 规则6：连接结合律
(E1 ⋈ E2) ⋈ E3 ≡ E1 ⋈ (E2 ⋈ E3)
```

深度分析 ：连接结合律允许我们改变连接顺序，这对性能有重大影响。



规则7：选择在连接上的分配律

当选择条件θ1只涉及E1的属性，θ2只涉及E2的属性时：

```
σ_{θ1∧θ2}(E1 ⋈_θ E2) ≡ σ_{θ1}(E1) ⋈_θ σ_{θ2}(E2)
```

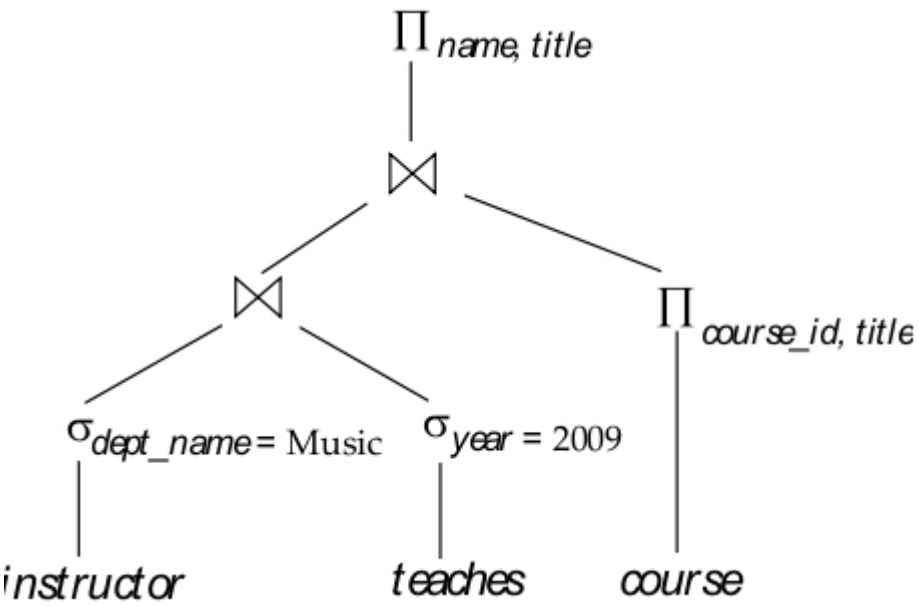
例题2：选择分配律应用

查询：计算机系教师2017年所教课程

```
SELECT * FROM instructor, teaches
WHERE instructor.ID = teaches.ID
AND dept_name = 'Comp. Sci.' AND year = 2017;
```

应用规则7重写：

```
σ_{dept_name='Comp. Sci.' ∧ year=2017}(instructor ⋈ teaches)
≡ σ_{dept_name='Comp. Sci.'}(instructor) ⋈ σ_{year=2017}(teaches)
```



规则8：投影在连接上的分配律

复杂但重要，允许提前消除不需要的属性。

三、启发式优化深度分析

3.1 启发式优化策略

基于经验规则，不依赖成本估计：

1. **尽早执行选择操作**
2. **尽早执行投影操作**
3. **先执行最限制性的操作**
4. **优先执行结果集较小的连接**

3.2 启发式优化步骤详解

步骤1：分解合取谓词

将复杂条件分解为简单条件，便于优化器处理。

例题3：复杂谓词分解

```
SELECT ARTIST.NAME
FROM ARTIST, APPEARS, ALBUM
WHERE ARTIST.ID = APPEARS.ARTIST_ID
AND APPEARS.ALBUM_ID = ALBUM.ID
AND ALBUM.NAME = "Andy's OG Remix";
```

分解为三个独立条件，便于分别优化。

步骤2：谓词下推

将选择条件推到查询树的最底层。

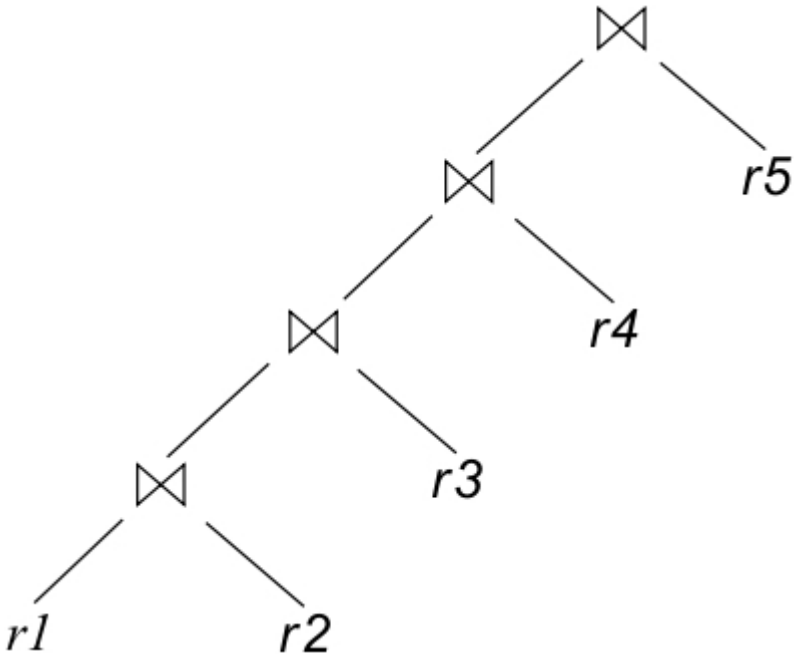
深度分析 ：谓词下推可以减少中间结果大小，是最重要的优化技术之一。

步骤3：用连接替换笛卡尔积

将笛卡尔积与选择条件合并为连接操作。

步骤4：投影下推

提前消除不需要的属性。



3.3 左深连接树

大多数数据库系统使用左深连接树（left-deep join trees），特点：

- 右输入总是基关系
- 便于流水线执行
- 减少优化复杂度

四、基于成本的优化深度解析

4.1 成本估计基础

成本估计基于统计信息：

- **关系级统计** ：元组数(n_r)、块数(b_r)、元组大小(l_r)
- **属性级统计** ：不同值数V(A, r)、选择基数SC(A, r)

4.2 选择操作大小估计

等值选择

```
SC(A = constant) = n_r / V(A, r)
```

范围选择

```
SC(A ≥ a) ≈ n_r × (max(A) - a + 1) / (max(A) - min(A) + 1)
```

复杂选择

- **合取选择** ：假设条件独立，选择性相乘
- **析取选择** ：使用概率公式
- **否定选择** ：n_r - 满足条件的元组数

例题4：选择基数计算

关系people有5个元组，age属性有5个不同值(0-4)：

- SC(age=2) = 5/5 = 1
- SC(age≥2) ≈ 5×(4-2+1)/(4-0+1) = 5×3/5 = 3

4.3 连接操作大小估计

外键连接

如果R ∩ S是S的外键引用R，则：

```
size(r ⋈ s) = n_s
```

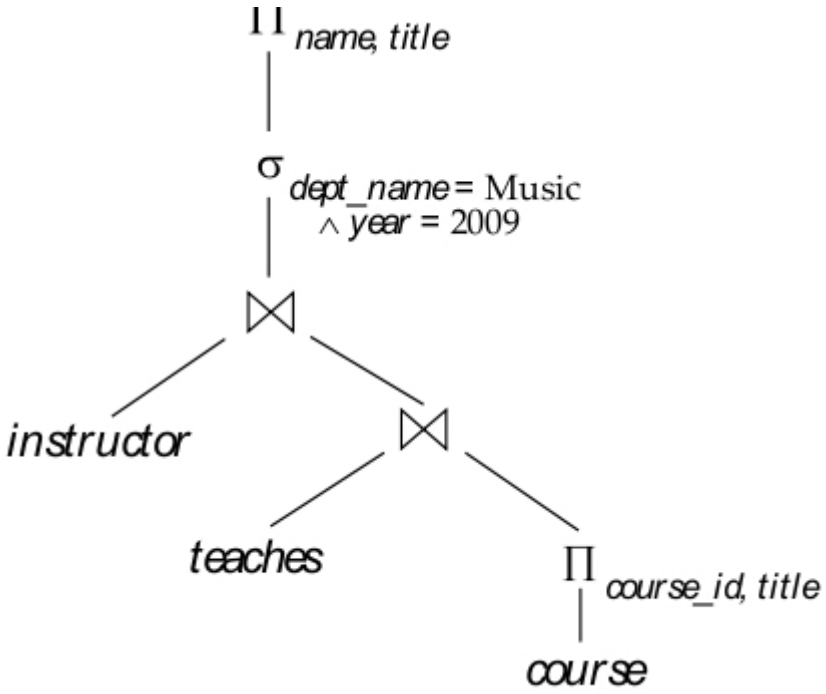
一般情况

```
size(r ⋈ s) = min(n_r × n_s / V(A, s), n_r × n_s / V(A, r))
```

例题5：连接大小估计

student ⋈ takes：

- n_student = 5000, n_takes = 10000
- V(ID, takes) = 2500, V(ID, student) = 5000
- 估计1：5000×10000/2500 = 20000
- 估计2：5000×10000/5000 = 10000
- 选择较小值：10000



4.4 动态规划连接顺序优化

问题复杂度

n个关系的连接有(2(n-1))!/(n-1)!种顺序：

- n=7 → 665,280种
- n=10 → 超过1760亿种

动态规划算法

```
procedure findbestplan(S):
    if bestplan[S].cost ≠ ∞:
        return bestplan[S]

    if |S| == 1:
        # 单关系最佳访问路径
        bestplan[S] = 最佳单关系计划
    else:
        for each non-empty proper subset S1 of S:
            P1 = findbestplan(S1)
```

```
            P2 = findbestplan(S - S1)
            for each join algorithm A:
                cost = P1.cost + P2.cost + join_cost(A, P1, P2)
                if cost < bestplan[S].cost:
                    bestplan[S] = 使用算法A连接P1和P2的计划

    return bestplan[S]
```

时间复杂度

- 左深树：O(n × 2^n)
- 茂密树：O(3^n)
- n=10时：约59,000 vs 1760亿

例题6：动态规划实例

优化r1 ⋈ r2 ⋈ r3 ⋈ r4的连接顺序：

1. 计算所有单关系计划成本
2. 计算所有两关系连接成本
3. 计算所有三关系连接成本
4. 计算四关系连接成本

通过子问题重用大幅减少计算量。

五、统计信息和直方图

5.1 统计信息类型

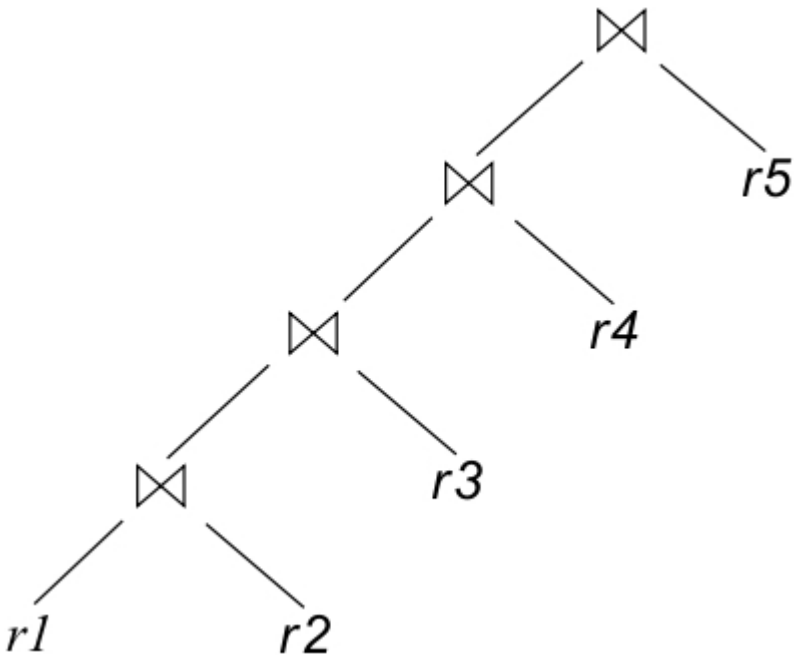
数据库系统维护的统计信息：

- 基本统计：n_r, b_r, f_r, V(A, r)
- 高级统计：直方图、最频繁值、相关系数

5.2 直方图技术

等深直方图

将值域划分为多个区间，每个区间包含近似相同数量的元组。



例题7：直方图应用

age属性值域[0,19]，建立4个桶的等深直方图：

- 桶1: [0,4] - 包含25%的元组
- 桶2: [5,8] - 包含25%的元组
- 桶3: [9,14] - 包含25%的元组
- 桶4: [15,19] - 包含25%的元组

估计age ≥ 10的选择性：桶3的部分 + 桶4全部。

六、高级优化技术

6.1 嵌套子查询优化

问题分析

相关子查询对每个外层元组执行一次，效率极低。

例题8：嵌套子查询优化

```
SELECT name FROM instructor
WHERE EXISTS (
    SELECT * FROM teaches
    WHERE instructor.ID = teaches.ID AND teaches.year = 2019
);
```

优化策略：去相关化

转换为半连接：

```
SELECT name FROM instructor
WHERE ID IN (SELECT ID FROM teaches WHERE year = 2019);
```

或者使用连接重写：

```
SELECT DISTINCT instructor.name
FROM instructor, teaches
WHERE instructor.ID = teaches.ID AND teaches.year = 2019;
```

6.2 物化视图维护

增量视图维护

当基关系更新时，只计算视图的增量变化。

连接视图的增量维护 ：

- 插入：$v_new = v_old \cup (i_r \bowtie s)$
- 删除：$v_new = v_old - (d_r \bowtie s)$

聚合视图的增量维护

维护计数和总和，而不是直接重新计算。

6.3 多查询优化

同时优化多个查询，共享公共子表达式。

例题9：多查询优化

```
-- 查询1
SELECT * FROM (r NATURAL JOIN t) NATURAL JOIN s;

-- 查询2
SELECT * FROM (r NATURAL JOIN u) NATURAL JOIN s;
```

优化策略：先计算r ⋈ s，然后分别与t和u连接。

七、综合例题与真题分析

7.1 综合优化例题

例题10：复杂查询优化

给定数据库模式：

- student(sid, name, dept)
- takes(sid, cid, grade)
- course(cid, title, credits)

查询：找出计算机系学生所选课程的课程名和成绩，按成绩降序排列，只显示前10名。

优化步骤 ：

1. **选择下推** ：先选择dept = 'Comp. Sci.'的学生
2. **连接顺序选择** ：分析表大小和索引情况
3. **投影下推** ：尽早消除不需要的属性
4. **排序优化** ：考虑使用索引避免全排序

7.2 真题风格练习题

练习题1：连接顺序选择

关系R(A,B)、S(B,C)、T(C,D)的统计信息：

- n_R = 1000, n_S = 10000, n_T = 100
- V(B,R) = 500, V(B,S) = 1000, V(C,S) = 200, V(C,T) = 50

查询：R ⋈ S ⋈ T

要求 ：

1. 估计每种连接顺序的中间结果大小
2. 选择最优连接顺序
3. 计算总代价估计

解答步骤 ：

1. 计算R ⋈ S的大小：min(1000×10000/500, 1000×10000/1000) = 10000
2. 计算S ⋈ T的大小：min(10000×100/200, 10000×100/50) = 5000
3. 比较不同连接顺序的中间结果大小
4. 选择产生最小中间结果的顺序

练习题2：索引选择优化

查询：

```
SELECT * FROM employee
WHERE dept = 'Engineering' AND salary > 100000 AND age < 40;
```

可用索引：

- 索引1：dept上的B+树索引（非聚簇）
- 索引2：salary上的B+树索引（聚簇）
- 索引3：age上的哈希索引

要求：

1. 分析每种索引策略的代价
2. 选择最优索引组合
3. 考虑复合索引的可能性

八、实际数据库系统的优化特性

8.1 优化器提示

大多数数据库系统支持优化器提示（hints）：

```
SELECT /*+ INDEX(employee emp_salary_idx) */ *
FROM employee WHERE salary > 50000;
```

8.2 执行计划查看

使用EXPLAIN命令查看查询执行计划：

```
EXPLAIN SELECT * FROM instructor WHERE salary > 50000;
```

8.3 自适应查询处理

现代数据库系统支持运行时优化调整：

- 根据实际执行统计调整计划
- 应对统计信息不准确的情况
- 处理数据倾斜问题

九、总结与学习建议

9.1 重点知识总结

1. **等价规则**：优化基础，允许安全的重写
2. **启发式优化**：基于经验规则快速优化
3. **成本估计**：基于统计信息预测执行代价
4. **动态规划**：高效搜索最优连接顺序
5. **高级技术**：嵌套查询、物化视图、多查询优化

9.2 学习建议

1. **理解原理**：掌握每个优化技术的理论基础
2. **实践分析**：使用EXPLAIN分析真实查询的执行计划
3. **参数调整**：了解统计信息收集和更新机制
4. **案例研究**：分析典型查询模式的优化策略

9.3 考试重点

根据真题模式，重点关注：

- 等价规则的应用
- 连接顺序选择
- 成本估计计算
- 实际优化策略选择

通过深入理解这些概念并结合实际例题练习，可以全面掌握查询优化这一重要主题。